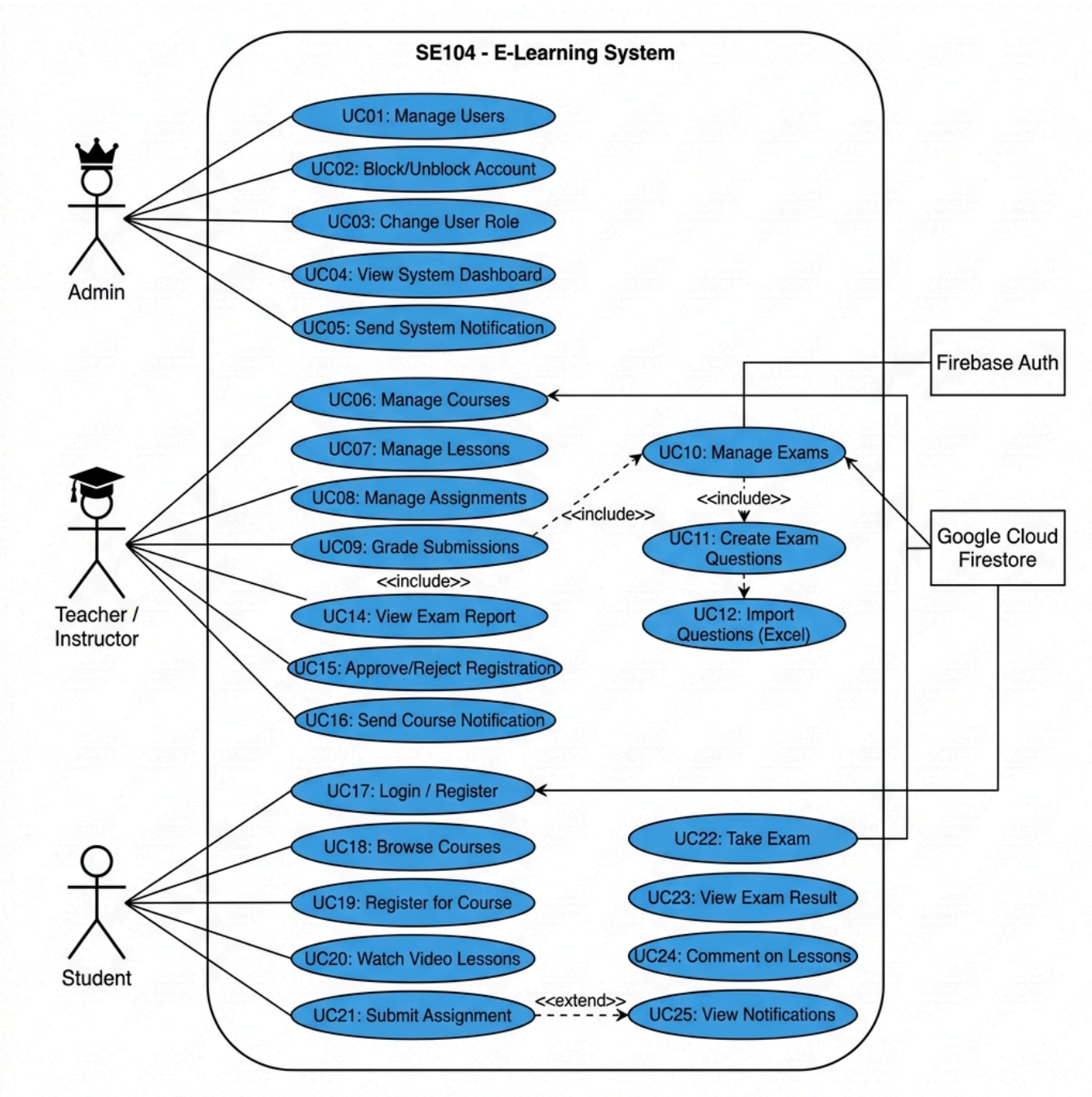


USE CASE SPECIFICATION

Hệ Thống E-Learning — SE104

Version:	1.0
Date:	2026-05-27
Tech Stack:	WPF Client · ASP.NET Core Web API · Google Cloud Firestore · Firebase Auth

1. Use Case Diagram



2. Danh Sách Actors

Actor	Mô Tả	Role trong hệ thống
Admin	Quản trị viên hệ thống	Quản lý user, xem thống kê, gửi thông báo toàn hệ thống
Teacher / Instructor	Giảng viên	Quản lý lớp học, bài giảng, bài tập, bài thi, duyệt đăng ký
Student	Học sinh / Sinh viên	Đăng ký khóa học, học bài, làm bài tập và bài thi
Firebase Auth	Hệ thống xác thực (External)	Xác thực Email/Password và Google OAuth
Firestore	CSDL NoSQL (External)	Lưu trữ toàn bộ dữ liệu hệ thống

3. Danh Sách Use Cases

ID	Use Case	Actor Chính	Actor Phụ
UC01	Quản lý người dùng (Manage Users)	Admin	Firestore
UC02	Khóa / Mở khóa tài khoản	Admin	Firestore
UC03	Thay đổi vai trò người dùng	Admin	Firestore
UC04	Xem Dashboard hệ thống	Admin	Firestore
UC05	Gửi thông báo hệ thống	Admin	Firestore
UC06	Quản lý khóa học	Teacher	Firestore
UC07	Quản lý bài giảng (Video)	Teacher	Firestore
UC08	Quản lý bài tập	Teacher	Firestore
UC09	Chấm điểm bài nộp	Teacher	Firestore
UC10	Quản lý bài thi	Teacher	Firestore
UC11	Tạo câu hỏi thủ công	Teacher	Firestore
UC12	Import câu hỏi từ Excel	Teacher	ClosedXML
UC13	Publish / Unpublish bài thi	Teacher	Firestore
UC14	Xem báo cáo kết quả thi	Teacher	Firestore
UC15	Duyệt / Từ chối đăng ký	Teacher	Firestore
UC16	Gửi thông báo lớp học	Teacher	Firestore
UC17	Đăng nhập / Đăng ký	Student	Firebase Auth, Firestore

ID	Use Case	Actor Chính	Actor Phụ
UC18	Tìm kiếm khóa học	Student	Firestore
UC19	Đăng ký khóa học	Student	Firestore
UC20	Xem bài giảng (Video)	Student	Firestore
UC21	Nộp bài tập	Student	Firestore
UC22	Làm bài thi	Student	Firestore
UC23	Xem kết quả thi	Student	Firestore
UC24	Bình luận bài giảng	Student	Firestore
UC25	Xem thông báo	Student	Firestore

4. Quan Hệ Giữa Use Cases

Quan Hệ	Use Case Nguồn	Use Case Đích	Loại
UC10 → UC11	Quản lý bài thi	Tạo câu hỏi thủ công	<<include>>
UC11 → UC12	Tạo câu hỏi	Import từ Excel	<<extend>>
UC22 → UC23	Làm bài thi	Xem kết quả thi	<<include>>
UC19 → UC15	Đăng ký khóa học	Duyệt đăng ký	<<include>>
UC17 → UC18	Đăng nhập	Tìm kiếm khóa học	<<extend>>
UC08 → UC09	Quản lý bài tập	Chấm điểm	<<include>>

5. Use Case Specifications Chi Tiết

UC17 — Đăng Nhập / Đăng Ký

Trường	Nội Dung
Use Case ID	UC17
Use Case Name	Đăng Nhập / Đăng Ký
Actor	Student, Teacher, Admin
Description	Người dùng xác thực danh tính qua Email/Password hoặc Google OAuth. Hệ thống cấp JWT token nội bộ sau khi xác thực thành công.

Trường	Nội Dung
Preconditions	Ứng dụng WPF đã khởi động. Có kết nối Internet.
Trigger	Người dùng mở ứng dụng hoặc hết phiên đăng nhập.

Luồng chính (Email/Password):

1. Người dùng nhập Email và Password → Click "Đăng nhập"
2. WPF validate input (không rỗng, định dạng email hợp lệ)
3. WPF gọi `POST /api/Auth/login { Email, Password }`
4. API gọi Firebase Auth `signInWithPassword`
5. Firebase xác thực thành công → trả về `localId, idToken`
6. API kiểm tra `users/{localId}` trong Firestore: `IsBlocked?`
7. API tạo JWT nội bộ (HS256, expiry 24h) với claims `{ UID, Email, Role }`
8. WPF lưu token vào `ApiService`, điều hướng theo Role

Luồng thay thế (Google OAuth):

1. Người dùng click "Đăng nhập với Google"
2. WPF mở Google OAuth popup → nhận `id_token`
3. WPF gọi `POST /api/Auth/Login-google { FirebaseToken }`
4. API gọi Firebase IDP `signInWithIdp`
5. Nếu user chưa tồn tại → tự động tạo tài khoản với Role = "Student"
6. Tiếp tục từ bước 7 của luồng chính

Luồng ngoại lệ:

- Email/Password sai → `401 "Tên đăng nhập hoặc mật khẩu không chính xác"`
- Tài khoản bị khóa → `400 "Tài khoản đã bị khóa bởi Admin"`
- Mạng lỗi → `CustomDialog.Error`

Postconditions	JWT token được lưu. Dashboard theo Role được hiển thị.
Business Rules	Mỗi Role được điều hướng tới MainWindow khác nhau. <code>IsBlocked = true</code> chặn đăng nhập.

UC19 — Đăng Ký Khóa Học

Trường	Nội Dung
Use Case ID	UC19
Use Case Name	Đăng Ký Khóa Học

Trường	Nội Dung
Actor	Student (chính), Teacher (phê duyệt)
Description	Học sinh gửi yêu cầu đăng ký vào một khóa học. Giáo viên xem xét và duyệt hoặc từ chối.
Preconditions	Học sinh đã đăng nhập. Khóa học đang tồn tại và hoạt động.
Trigger	Học sinh click "Đăng ký khóa học" trên màn hình khóa học.

Luồng chính:

1. Học sinh chọn khóa học từ danh sách
2. Hệ thống kiểm tra trạng thái đăng ký hiện tại
3. Nếu chưa đăng ký → hiển thị nút "Đăng ký"
4. Học sinh click → `CustomDialog.confirm`
5. WPF gọi `POST /api/Courses/{courseId}/register`
6. API kiểm tra Course tồn tại trong Firestore
7. API tạo document: `courseRegistrations/{uid}_{courseId}` với `status = "pending"`
8. Học sinh thấy badge "Đang chờ duyệt"
9. Giáo viên mở tab "Học Sinh" → `GET /api/Courses/{courseId}/students`
10. Giáo viên click "Duyệt" → `PUT .../approve`
11. Firestore cập nhật `status = "accepted"`, `approvedDate = ServerTimestamp`
12. Học sinh có thể truy cập bài học, bài thi của lớp

Luồng thay thế — Từ chối:

- Giáo viên click "Từ chối" → `PUT .../reject`
- Firestore: `status = "rejected"`
- Gửi thông báo từ chối đến học sinh

Luồng ngoại lệ:

- Khóa học không tồn tại → `404`
- Đã đăng ký rồi → hiển thị trạng thái hiện tại

Postconditions	Registration document tồn tại với <code>status ∈ {pending, accepted, rejected}</code> .
Business Rules	Document ID = <code>"{uid}_{courseId}"</code> đảm bảo mỗi học sinh chỉ có 1 registration/khóa học. Chỉ <code>accepted</code> mới cho phép xem bài thi.

UC22 — Làm Bài Thi

Trường	Nội Dung
Use Case ID	UC22
Use Case Name	Làm Bài Thi (Take Exam)
Actor	Student
Description	Học sinh vào làm bài thi với đếm ngược thời gian, auto-save draft, và nộp bài. Hệ thống tự động chấm điểm trắc nghiệm.
Preconditions	Học sinh đã đăng nhập và được duyệt vào lớp. Bài thi đã được Publish (<code>IsPublished = true</code>). Còn trong số lần làm tối đa (<code>MaxAttempts</code>).
Trigger	Học sinh click "Vào làm bài" trong StudentQuizView.

Luồng chính:

1. WPF tải danh sách bài thi: `GET /api/Exams/my-exams`
2. API query các khóa học đã được `accepted` → filter bài thi `IsPublished = true`
3. Học sinh chọn bài thi → `GET /api/Exams/{examId}`
4. Hệ thống kiểm tra draft: `GET /api/Exams/{examId}/drafts/{studentId}`
5. **Nếu có draft:** Khôi phục câu trả lời, tính thời gian còn lại từ `StartedAt`
6. **Nếu không có draft:** Khởi tạo mới, bắt đầu countdown từ `TimeLimitMinutes`
7. Học sinh trả lời câu hỏi (trắc nghiệm, tự luận)
8. Auto-save draft mỗi 30 giây: `POST /api/Exams/drafts`
9. Học sinh click "Nộp bài" → `CustomDialog.Confirm`
10. WPF gọi `POST /api/Exams/{examId}/submit { Answers: { "0":1, "1":2, ... } }`
11. API chấm điểm: `percentage = (earnedPoints/totalPoints) * 100`, `score = percentage / 10`
12. API lưu vào `exam_submissions`
13. API xóa draft: `DELETE /api/Exams/{examId}/drafts/{studentId}`
14. Hiển thị `QuizResultDetailView`

Luồng thay thế — Hết giờ:

- Timer countdown = 0 → Auto-submit không cần xác nhận
- Hiển thị thông báo "Hết thời gian! Bài thi đã được nộp tự động"

Luồng ngoại lệ:

- Mất kết nối trong khi thi → draft được lưu local, tiếp tục khi có kết nối
- Exam không tồn tại → `404`, quay về danh sách

Postconditions	<code>exam_submissions</code> document được tạo. Draft bị xóa. Học sinh thấy kết quả điểm.
Business Rules	Score hệ 10 = Percentage / 10. Draft ID = <code>{examId}_{studentId}</code> . Answers được lưu theo index câu hỏi.

UC10 — Quản Lý Bài Thi (Create Exam)

Trường	Nội Dung
Use Case ID	UC10
Use Case Name	Quản Lý & Tạo Bài Thi
Actor	Teacher / Instructor
Description	Giáo viên tạo bài thi 2 bước: (1) nhập thông tin tổng quan, (2) thêm câu hỏi thủ công hoặc import Excel.
Preconditions	Giáo viên đã đăng nhập với Role = "Instructor". Có ít nhất 1 khóa học đang dạy.
Trigger	Giáo viên click "Tạo bài thi mới" trong ExamManagementView.

Luồng chính — Bước 1 (CreateExamView):

- Giáo viên chọn lớp học từ ComboBox (load từ API)
- Nhập: Tên bài thi, Mô tả, Thời gian, Điểm đạt, Hạn nộp
- Cài đặt: IsPublished, AllowReview, RandomizeQuestions, MaxAttempts
- Preview card cập nhật real-time
- Click "Tiếp theo" → Validate → Navigate đến CreateExamQuestionsView

Luồng chính — Bước 2 (CreateExamQuestionsView):

- Giáo viên thêm câu hỏi thủ công: Content + 4 Options + Đáp án đúng + Điểm
- Hoặc Import Excel (.xlsx): kéo thả / chọn file → parse từng dòng → validate
- Click "Hoàn Tất" → `POST /api/Exams { CreateExamRequest }`
- API lưu: exam document + Questions[] inline + QuestionIds[] (tương thích ngược)
- Nếu `IsPublished = true` → gửi thông báo đến toàn lớp
- Navigate về ExamManagementView

Luồng ngoại lệ:

- Không có câu hỏi nào → Warning "Thêm ít nhất 1 câu hỏi"
- File Excel không hợp lệ → bỏ qua dòng lỗi, báo số câu bị bỏ
- Quay lại từ Bước 2 → Bước 1 tự restore thông tin đã nhập

Postconditions	Exam document được tạo trong Firestore với <code>Questions[]</code> inline. Nếu <code>IsPublished = true</code> , học sinh nhận thông báo.
Business Rules	<code>QuestionId = Guid.NewGuid("N")</code> . Document có cả <code>TimeLimitMinutes</code> và <code>DurationMinutes</code> để tương thích ngược.

UC15 — Duyệt / Từ Chối Đăng Ký

Trường	Nội Dung
Use Case ID	UC15
Use Case Name	Duyệt / Từ Chối Đăng Ký
Actor	Teacher / Instructor
Description	Giáo viên xem danh sách học sinh đã gửi yêu cầu đăng ký và quyết định chấp nhận hoặc từ chối từng yêu cầu.
Preconditions	Giáo viên đã đăng nhập. Có ít nhất 1 yêu cầu <code>status = "pending"</code> trong lớp.
Trigger	Giáo viên mở tab "Học Sinh" trong CourseDetailView.

Luồng chính — Duyệt:

- `GET /api/Courses/{courseId}/students` → trả về danh sách + JOIN FullName/Email
- Giáo viên chọn học sinh → click "Duyệt"
- `PUT /api/Courses/{courseId}/registrations/{regId}/approve`
- Firestore: `status = "accepted"`, `approvedDate = ServerTimestamp`
- Học sinh được thêm vào lớp, nhận thông báo

Luồng thay thế — Từ chối:

- Giáo viên click "Từ chối"
- `PUT /api/Courses/{courseId}/registrations/{regId}/reject`
- Firestore: `status = "rejected"`
- Học sinh nhận thông báo từ chối

Luồng thay thế — Xóa khỏi lớp:

- Giáo viên click "Xóa" (đã accepted)
- `DELETE /api/Courses/{courseId}/registrations/{regId}`
- Firestore: DELETE document
- `StudentCount -= 1` trên Course document

Postconditions	Registration status được cập nhật. Học sinh được/không được truy cập nội dung lớp học.
Business Rules	Chỉ <code>status = "accepted"</code> cho phép truy cập bài thi qua <code>my-exams</code> query.

UC02 — Khóa / Mở Khóa Tài Khoản

Trường	Nội Dung
Use Case ID	UC02
Use Case Name	Khóa / Mở Khóa Tài Khoản
Actor	Admin
Description	Admin thay đổi trạng thái <code>IsBlocked</code> của tài khoản người dùng, ngăn hoặc cho phép đăng nhập.
Preconditions	Admin đã đăng nhập. User cần khóa đang tồn tại trong hệ thống.
Trigger	Admin click toggle "Khóa tài khoản" trong AdminUsersView.

Luồng chính:

- Admin tìm kiếm / lọc user trong danh sách
- Admin click nút "Khóa" (hoặc "Mở khóa")
- `CustomDialog.Confirm` "Bạn có chắc muốn khóa tài khoản này?"
- WPF gọi `PUT /api/Users/{userId} { IsBlocked: true }`
- Firestore UPDATE `Users/{userId}.IsBlocked = true`
- Danh sách cập nhật, user hiển thị badge "Đã khóa"

Luồng ngoại lệ:

- User không tồn tại → `404`
- Cố khóa tài khoản Admin khác → cần xác nhận thêm

Postconditions	<code>Users/{userId}.IsBlocked</code> được cập nhật. Lần đăng nhập tiếp theo của user bị chặn.
Business Rules	Logic chặn đăng nhập được thực thi ở API: sau Firebase xác thực thành công, API kiểm tra <code>IsBlocked</code> trước khi cấp JWT.

UC09 — Chấm Điểm Bài Nộp

Trường	Nội Dung
Use Case ID	UC09
Use Case Name	Chấm Điểm Bài Nộp (Grade Submission)
Actor	Teacher / Instructor
Description	Giáo viên xem các bài nộp của học sinh cho một bài tập và nhập điểm + nhận xét.
Preconditions	Bài tập đã tạo. Học sinh đã nộp ít nhất 1 bài. Giáo viên có quyền với lớp đó.
Trigger	Giáo viên click "Xem bài nộp" trong danh sách bài tập.

Luồng chính:

1. `GET /api/Courses/{courseId}/assignments/{asmId}/submissions`
2. Giáo viên xem danh sách bài nộp (FileUrl, SubmittedAt, IsLate)
3. Giáo viên nhập Score và Comment
4. `PUT /api/Courses/{courseId}/assignments/{asmId}/submissions/{studentId}/grade { Score, Comment }`
5. Firestore UPDATE `Score`, `Comment` trong submission document
6. Giáo viên click "Công bố điểm": `PUT .../publish-grades`
7. Firestore UPDATE `Assignments/{asmId}.IsGradesPublished = true`
8. Học sinh thấy điểm trong StudentCourseView

Luồng ngoại lệ:

- Submission không tồn tại → `404`
- Score không hợp lệ (`< 0` hoặc `> 10`) → validation error

Postconditions	<code>Score</code> và <code>Comment</code> được lưu. Nếu <code>IsGradesPublished = true</code> , học sinh thấy điểm.
Business Rules	Mỗi học sinh có 1 submission document (<code>DocumentId = studentId</code>). Điểm chỉ hiển thị khi <code>IsGradesPublished = true</code> .

6. Ma Trận Actor — Use Case

Use Case	Admin	Teacher	Student
UC01 Manage Users	✓		
UC02 Block Account	✓		
UC03 Change Role	✓		

Use Case	Admin	Teacher	Student
UC04 Dashboard	✓		
UC05 System Notification	✓		
UC06 Manage Courses		✓	
UC07 Manage Lessons		✓	
UC08 Manage Assignments		✓	
UC09 Grade Submissions		✓	
UC10 Manage Exams		✓	
UC11 Create Questions		✓	
UC12 Import Excel		✓	
UC13 Publish Exam		✓	
UC14 Exam Report		✓	
UC15 Approve/Reject		✓	
UC16 Course Notification		✓	
UC17 Login/Register	✓	✓	✓
UC18 Browse Courses			✓
UC19 Register Course			✓
UC20 Watch Lessons			✓
UC21 Submit Assignment			✓
UC22 Take Exam			✓
UC23 View Result			✓
UC24 Comment			✓
UC25 View Notifications	✓	✓	✓

7. Lịch Sử Phiên Bản

Phiên Bản	Ngày	Mô Tả
1.0	2026-05-27	Tạo lần đầu — 25 Use Cases, 3 Actors